MUMBAI PORT AUTHORITY

BUILDING PERMISSION PORTAL

Engineering Handoff & Technical Decision Record

Problem Analysis, Architecture, Stack Decisions, and Implementation Roadmap

Document Type: Engineering Handoff & Technical Decision Record

Project: MbPA Building Permission Portal — Rebuild

Domain: GovTech / E-Governance / Port-Sector Special Planning Authority

Version: 1.0 — Complete Design-Phase Edition

Intent

This document gives its reader complete understanding of every engineering decision made on this project — the problem analysed, the options weighed, the alternatives rejected, and the reasoning behind every final call. It is the complete information pool from which implementation, audit, and stakeholder handoff all draw.

Table of Contents

1. Executive Summary

Mumbai Port Authority (MbPA) operates as a Special Planning Authority under the Major Port Authorities Act, 2021, with statutory responsibility for the full building-permission lifecycle on port land — from initial application through Occupancy Certificate. A working prototype (HTML + Google Apps Script) already exists, built by a domain-expert intern without a software-engineering background. This document is the complete engineering handoff for rebuilding that prototype into a production-oriented system: it captures the problem being solved, the prototype's specific failures, every architecture and technology decision made in response, and the full data model that implements those decisions.

Three things distinguish this effort from a typical internal tooling project. First, it is government software handling Aadhaar-linked personal data, carrying personal criminal liability under the Aadhaar Act for mishandling — not an abstract compliance checkbox. Second, it must remain maintainable on a multi-decade horizon, since government systems routinely outlive the engineers who built them. Third, every infrastructure decision was made under a real zero-confirmed-budget constraint, which shaped the stack as much as any technical criterion did.

> **DECISION:** **Every decision in this document was reached by enumerating real alternatives, evaluating them against explicit, named criteria, and recording why the losing options lost — not by defaulting to the most familiar or most popular choice.**

2. Problem Statement

2.1 The National Backlog Behind a Single Portal

India's construction-permitting regime has historically been one of the most cited regulatory bottlenecks in the country's investment climate. Under the World Bank's former Doing Business “Dealing with Construction Permits” indicator (discontinued globally on 16 September 2021, but the most recent measurements remain the only systematic cross-country benchmark available), India ranked 183rd of 189 economies in 2015 — among the worst performers in the world on this specific dimension — before a sustained reform push moved it to 52nd by 2019, a jump of 129 ranks in four years.

The reforms behind that jump are directly relevant to this project: the time to obtain a construction permit in Mumbai fell from 128.5 days to 99 days, and in Delhi from 157.5 to 91 days, driven specifically by the introduction of Online Building Plan Approval Systems and Common Application Forms for digital submission. The mechanism that improved India's standing was exactly the kind of system this project is building — not a policy change, a software change.

The same body of research notes a starker number behind the formal statistics: in developing economies generally, an estimated 60–80% of construction projects proceed without formal approval at all, because the cost and delay of the formal process creates a direct incentive to bypass it. A permission system's design — its speed, its transparency, its predictability — is not a UX nicety in this context. It is a meaningful lever on whether construction happens inside or outside the regulated system at all.

Sources: World Bank Doing Business archive (discontinued 2021); SRCC “Construction Permit Enablers in India”; Sharma, “Ease of Getting Construction Permits in India: Cross-Country Comparison with World’s Best” (2017).

2.2 MbPA's Specific Position

MbPA is not a municipal corporation — it is a Special Planning Authority under the Major Port Authorities Act, 2021, governing port land specifically, using its own regulatory framework (UPDR-2026) and its own terminology (PLPN, not CTS No.; PMP, not Development Plan; PRL, not R.L. of street). This matters because MbPA cannot simply adopt a municipal system wholesale — every comparable system examined in Section 3 is built for municipal or state jurisdiction, not for a port authority's specific statutory position, land-use constraints (CRZ, operational port zones, heritage buffers), and terminology.

The starting point for this rebuild is a working prototype that already digitises the core lifecycle — application intake, milestone tracking, officer review, fee calculation, certificate issuance — built in HTML and Google Apps Script by a domain-expert intern. The prototype is a genuine asset: it encodes real domain knowledge (the milestone chain, the fee formula, the document-slot structure) that would otherwise have to be learned from scratch. It is also, as Section 2.3 details, carrying specific defects that make it unsuitable for production as-is.

2.3 What the Prototype Actually Does Wrong

A line-by-line reconciliation of the prototype's source (Code.gs, index.html) against its own PRD surfaced eight concrete defects, several of which are not minor bugs but compliance or security failures:

Defect	What it does	Why it matters	Disposition
Full Aadhaar storage	Stores complete Aadhaar numbers in Script	Confirmed compliance defect under the Aadhaar	Must be remediated before any production

	Properties and a Drive JSON mirror	Act — carries personal criminal liability (§§37/38/42), not merely a best-practice gap	deployment — see §6
Stream & Fee Planner gated behind login	Prototype requires sign-in; PRD states it should be public, pre-account	Contradicts the system's own stated Ease-of-Doing-Business purpose	Resolved: made public in the rebuild
Fee-rule duplication	Identical fee constants hand-copied into both backend and frontend code	Two independently-editable copies of the same regulatory numbers is a drift risk the system's own design principle (UPDR-2026 as single source of truth) would itself flag	Resolved: single-sourced via a versioned configuration table — see §8
Undocumented 7-question NOC wizard	A conditional-clearance question wizard (Railway, CRZ, MHCC heritage, AAI aviation, MPCB) with a benchmark-comparison engine exists in code with zero mention in the PRD	A materially richer concession-detection mechanism than the PRD describes was running in production-bound code without specification	Documented and carried into the rebuild — see §8
Demo placeholder benchmarks	FSI, open-space, height and setback benchmarks are hardcoded “(demo)” values, explicitly labelled as such in the UI	If carried forward unnoticed, every fee and concession calculation in production would be based on placeholder numbers	Externalised to configuration, flagged as pending real UPDR-2026 values — never silently reused
IOD auto-coupled to every rejection	Every officer rejection automatically generates a formal Intimation of Disapproval	PRD's own glossary implies IOD should be a distinct, discretionary action, not an automatic side-effect	Carried forward as an explicit open decision pending MbPA officer-workflow input
Hardcoded SLA toast bug	Rejection notice always displays “7 days” regardless of the milestone's actual SLA (21/30/15/7 days vary)	Cosmetic, but actively misleads officers about the real correction deadline	Resolved: values now single-sourced from milestone definitions, cannot drift
No session/security hardening	Hardcoded officer credentials in source; no formal secrets management	Direct credential-exposure risk in source control	Resolved — see §9

None of these defects diminish the prototype's value as a specification of the actual lifecycle — they are precisely the kind of finding a proper engineering handoff exists to surface before they are carried silently into production.

3. Comparative Landscape

Three Indian digital building-permission systems pre-date this project and were examined directly — not as competitors, since none of them serve port land or operate under UPDR-2026, but as the closest available evidence of what works and what fails at scale in this exact problem domain.

3.1 MCGM AutoDCR / OBPAS (Municipal Corporation of Greater Mumbai)

Launched in 2015, MCGM's Online Building Plan Approval System is the most mature comparable system examined, and the most directly relevant given its Mumbai jurisdiction.

Dimension	What AutoDCR Does	Relevance to This Project
Legal basis for IOD	Intimation of Disapproval issued under Section 346 of the Mumbai Municipal Corporation Act, 1888; valid 1 year, revalidatable up to 3 consecutive years	MbPA's own IOD concept is structurally identical — confirms IOD-as-distinct-instrument is the correct mental model, informing §8's certificate design
EODB Zero-FSI provision	Applicant may begin plinth work without waiting for concession approval, subject to a Comprehensive Undertaking and Indemnity	Direct precedent for treating speed-of-start as a first-class design goal, not just compliance
Completion timelines	Completion certificate granted within 7 working days of a request, after a single coordinated joint site visit across departments	Concrete evidence that multi-department coordination (the NOC wizard's railway/CRZ/heritage/aviation/pollution checks) can be collapsed into one inspection event rather than serialised
Digital signature requirement	Digitally signed PRC and 4A/4B documents required at submission; DSC-based signing built into the Architect/Licensed Surveyor console	Validates the DSC-centred certificate-issuance architecture adopted in §8 — this is the established norm, not an unusual requirement

3.2 TS-bPASS (Telangana State Building Permission Approval and Self-Certification System)

Launched by GHMC in February 2021 under the Telangana Municipalities Act, 2019, TS-bPASS is the most aggressive self-certification model examined, and the strongest evidence base for deemed-clearance design.

Dimension	What TS-bPASS Does	Relevance to This Project
Scale achieved	Over 1,00,000 construction approvals across Telangana's urban local bodies between November 2020 and December 2023; GHMC	Demonstrates that a fully online, self-certification-based permission system operates at real scale in an Indian state context within three

	alone accounted for 32,361 approvals	years of launch
Tiered instant approval	Plots up to 500 sq m and 10 m height receive instant approval via self-certification; larger/taller buildings go through a 21-day single-window review	Directly informed this project's deemed-clearance design — and its one deliberate exception
Deemed approval is revocable, not absolute	A deemed approval can be revoked by the Commissioner within 21 days if obtained through misrepresentation or in violation of building rules	Confirms that even the most aggressive deemed-clearance regime in India treats it as provisional, not as a substitute for verification — directly supporting this project's strongest single finding (§8.4): final occupancy/safety sign-off should never be silently auto-approved

3.3 MIDC BPAMS (Maharashtra Industrial Development Corporation)

MIDC's Building Plan Approval Management System mandates Digital Signature Certificate-only issuance for all building permission documents — no manual-signature fallback path exists in its design. This is the strictest DSC posture among the systems examined, and the direct reason this project's §8 explicitly retains a manual sign-print-scan-upload fallback rather than assuming every officer arrives with a working DSC token on day one: MIDC's all-or-nothing model is the right end-state, not necessarily the right starting point for a system going live without confirmed officer DSC provisioning.

3.4 What None of Them Address

All three systems are built for municipal or state industrial-development jurisdiction. None of them operate under a Special Planning Authority's specific statutory position, none handle CRZ/operational-port-zone/heritage-buffer exclusions as a first-class business rule, and none use MbPA's terminology (PLPN, PMP, PRL). This is not a gap any of them failed to close — it is simply outside their jurisdiction. It is, however, exactly the gap this project closes.

4. Proposed Solution — System Overview

The system is a rebuild, not an extension, of the existing prototype: the HTML + Google Apps Script implementation is replaced outright, while the domain logic it encodes — the milestone chain, the fee formula, the document-slot structure, the four-officer-role model — is carried forward deliberately, distinct from the defects in §2.3, which are not carried forward.

4.1 Governing Engineering Principle

Every architecture and stack decision in this document follows from one governing constraint: implementation velocity and minimal onboarding overhead are weighted heavily, because the project must move quickly with no pre-existing institutional infrastructure, and every infrastructure choice was evaluated against genuinely free options, not merely cheap ones, because no confirmed budget exists. This single constraint — not personal preference — is the throughline that explains why "boring," well-documented, low-ceremony technology was chosen at every layer in preference to more powerful but heavier alternatives.

4.2 Architectural Pattern: Modular Monolith

Microservices were never seriously considered. A project with this constraint profile, building for decades-scale maintainability, should produce one well-structured deployable unit — not a distributed system that requires its own orchestration discipline to operate safely. The system is a single Django + Django REST Framework (DRF) application, with a React frontend built and served as static files alongside it.

4.3 System Diagram

Applicant and officer browsers reach the system through a single same-domain reverse proxy, which routes /api/* to the Django/DRF application and everything else to the built React static files. This single routing decision eliminates an entire category of cross-origin configuration risk that a split-domain deployment would otherwise introduce.

Layer	Component
Entry point	Reverse proxy (Nginx-class) — single domain, routes /api/* vs static React build
Application	Django + Django REST Framework (Gunicorn) — session auth + CSRF
Database	Neon (managed PostgreSQL)
File/Object storage	Cloudflare R2 — uploaded documents, generated certificates
Scheduled work	Linux cron → manage.py run_sla_sweep (daily)
Certificate signing	Local DSC-token signing (officer-side) → signature verified on receipt
Email	Resend (transactional)
Internal ops tool	Django admin — separate access control,

	independent of the citizen/officer-facing UI

4.4 Background Processing Model

The system has exactly one genuinely recurring job (the daily SLA sweep) and one occasionally-slow task (certificate generation and signature verification). Both were evaluated against the alternative of a full task-queue infrastructure:

Option	Verdict	Reasoning
Celery + Redis	Rejected	Adds a separate stateful service to run, secure, and patch indefinitely, for a system with one recurring job — disproportionate to actual need
In-process scheduler (APScheduler)	Rejected	Risks the same job firing multiple times across multiple worker processes without added coordination logic — fragile for something as consequential as the SLA sweep
django-q2	Reserved as upgrade path	Lighter than Celery, can use the existing PostgreSQL database itself as the broker if real async needs emerge later
OS cron + management command	Selected	Zero new dependencies or services; `manage.py run_sla_sweep` triggered by plain Linux cron

Certificate generation and signature verification are handled synchronously, inline, within the request — at this system's actual volume (a handful of certificates per day for one port authority), there is no performance case yet for decoupling it.

5. Technology Stack — Decisions and Rationale

Every technology in this stack was selected by evaluating named alternatives against eight explicit criteria, not by defaulting to the most familiar option. The criteria, used consistently across every decision in this section: implementation velocity and ecosystem onboarding cost (C1); decades-stability and low ecosystem churn (C2); security-by-default posture (C3); gov.in/NIC hosting compatibility (C4); future-maintainer availability in Indian government/PSU IT (C5); ecosystem maturity for this system's specific needs — ORM, scheduled jobs, PDF/DSC signing, file uploads, email (C6); type safety (C7); and licensing cost (C8).

5.1 Backend Language: Python

Seven languages were considered: Java+Spring Boot, Python+Django, Node.js+Express, Go, .NET/C#, PHP+Laravel, and Ruby on Rails. Ruby, PHP, and .NET were eliminated in the first round — none offered a structural advantage over Python or Java strong enough to justify their onboarding cost or weaker future-maintainer pool in the Indian government-IT context. Go was eliminated despite best-in-class stability and security characteristics, because its ecosystem for this system's specific needs — DSC/PDF-signing libraries, Django-style admin scaffolding — is measurably thinner than Java's or Python's.

Node.js was eliminated next: Express provides no CSRF/XSS/SQLi protection by default, npm's supply-chain track record is documented as worse than PyPI's or Maven's (real incidents: event-stream, ua-parser-js, the colors.js/faker.js sabotage), and the “same language front-to-back” appeal is weaker than it first appears — a Node backend still pairs with a separate frontend build pipeline unless the application is fully server-rendered.

Criterion	Java + Spring Boot	Python + Django
C1	Slower — heavier ecosystem onboarding	Faster — flatter onboarding curve, lower setup ceremony
C2	Best of all candidates — JVM's backward-compatibility record is unmatched	Very strong — Django's explicit “boring tech” philosophy, stable since 2005
C3	Mature (Spring Security)	Mature (built-in CSRF/XSS/SQLi protection) — roughly equal
C5	Strongest — the de facto standard across NIC/PSU systems	Good and growing, not yet as entrenched as Java in this context
C6	Excellent (Spring Data JPA, Spring Scheduler, PDFBox/iText)	Excellent (Django ORM + admin scaffolding; pyHanko for PDF/DSC signing)

> **DECISION: Python. C1 is a present, binding constraint — implementation needs to move quickly. Java's C5 advantage is a future consideration, documented below as the named alternative should priorities shift.**

Java's onboarding cost is concrete, not a vague impression: JDK/JRE version management, choosing between Oracle JDK/OpenJDK/Temurin/Corretto distributions, and Maven's verbose XML configuration all add measurable setup ceremony beyond Python's install-and-run model. Modern tooling (SDKMAN!, Spring Initializr, IDE-managed builds) mitigates this significantly versus a decade ago, but does not

eliminate it. If implementation velocity were weighted less heavily — for instance, if MbPA's own IT function with existing Java staff took over development — Java + Spring Boot is the documented fallback choice.

5.2 Backend Framework: Django + Django REST Framework

Criterion	Django	Flask	FastAPI
C1	Wins — ORM, admin, auth, forms, templating included	Slower — assemble via extensions	Slower — assemble everything; no structural scaffolding
C3	Wins decisively — CSRF, auto-escaping, SQLi-safe ORM all on by default	Opt-in via extensions — easy to forget under delivery pressure	Same gap as Flask
C6	Wins — admin panel fits the officer console; scheduled-job integration for the SLA sweep	Workable but fully assembled, not given	Weakest — API-only framework
C7	Decent (type hints + django-stubs)	Decent, same as Django	Genuinely strongest — Pydantic validation

DECISION: Django + Django REST Framework, used as a pure API backend. Re-examined after the frontend architecture moved to a full SPA (§5.3), since Django's templating advantage became irrelevant once nothing is server-rendered — the admin panel survives as an internal MbPA ops tool regardless of what renders the citizen/officer-facing UI, an orthogonal benefit unaffected by that pivot. FastAPI's real advantages (Pydantic validation, auto-generated docs) were not decisive enough to give up Django's admin scaffolding and ORM maturity.

5.3 Frontend Architecture: Full Single-Page Application

The initial recommendation was server-rendered Django templates: this integrates natively without a separate API contract, requires no JavaScript build pipeline for the bulk of the application, and the system is mostly forms, tables, and status views rather than an app-like, real-time UI. This was reconsidered specifically to avoid the visual impression of a dated government portal.

The architecturally important correction made at this point: rendering mechanism does not determine visual quality. GOV.UK's own frontend and Basecamp/HEY are both server-rendered and look excellent — the dated-government-portal look comes from absent design systems and cluttered layout, not from server-side rendering itself. The more precise justification for the eventual SPA decision is that a mature component ecosystem (shadcn/ui, Radix, MUI) genuinely lowers the cost of a polished, app-like feel — not that React is inherently more capable of looking modern.

DECISION: Full SPA — React owns the entire UI; Django operates purely as an API. This was an informed choice, made only after the full security and operational overhead of the API+SPA split (§5.6) was laid out explicitly and accepted.

5.4 Frontend Framework: React

React, Preact, Angular, Vue, and Next.js were all evaluated. Under the narrower framing that existed before the full-SPA decision — React or Preact considered only for two embedded interactive widgets

within an otherwise server-rendered application — Preact won on bundle size (roughly 3kb versus React's 40kb+) and on not requiring a full build pipeline. Once the architecture moved to a full SPA, that framing no longer applied: bundle size matters far less than the depth of the component ecosystem needed for a polished, professional look, and React's ecosystem (shadcn/ui, Radix, MUI, Ant Design) is the deepest of any candidate for that specific goal.

Angular was examined on its own terms, not dismissed by default: its enforced structure and built-in routing/forms/dependency-injection are a genuine asset for multi-developer long-term maintenance. It did not win here because the deciding criterion — component/design-system ecosystem depth for the project's specific visual-quality goal — does not shrink for Angular regardless of team size; React's edge on that specific axis stands independent of how many people maintain the codebase.

Next.js — eliminated on independent merits, regardless of the SPA-versus-SSR question

Criterion	Finding
C2 (decades-stability)	Poor — the Pages Router to App Router transition was a major breaking architectural shift; Next.js has had several such disruptions, a worse churn signal than even comparable ecosystem incidents elsewhere
C4 (hosting compatibility)	Next.js's full feature set assumes a Vercel-tuned Node runtime; mapping that onto NIC/MeitY-empanelled infrastructure is unproven
Operational footprint	Pairing it with Django means running two separate server runtimes for one application — doubling the patch/monitor/security surface for the team maintaining it
Does it solve a real problem here?	No — Next.js's core value (server-side rendering for SEO) doesn't apply to a system that is almost entirely behind authentication

5.5 Design System: Tailwind CSS + shadcn/ui

The actual lever for “modern, sleek, professional” is not React itself — React with no styling discipline looks exactly as dated as a poorly designed server-rendered page would. Tailwind CSS plus shadcn/ui (or Radix) is the concrete tool selected for the visual-quality goal, layered on top of the React decision.

5.6 Security Implications of the API + SPA Split

Moving from a server-rendered Django application to a Django-API-plus-React-SPA split converted several things Django previously handled automatically into explicit engineering responsibilities. This list was produced deliberately before the SPA decision was finalised, so the decision was made with the real cost visible, not discovered after the fact.

Now explicit work	Detail
CORS	Did not exist under same-origin server rendering. Mitigated by the same-domain reverse-proxy topology (§4.3), avoiding cross-origin configuration and the risk of a misconfigured wildcard-plus-credentials setup
CSRF for session auth	React must explicitly fetch and attach the CSRF header; nothing does this automatically
API contract drift	Frontend and backend can now diverge; drf-

	spectacular generates an OpenAPI schema as the documented, code-derived contract
Rate limiting on auth/OTP endpoints	Now bare, scriptable API endpoints — mitigated with DRF throttling classes
Dependency surface doubles	Python backend and npm frontend dependencies both need tracking; the npm supply-chain risk that helped eliminate Node as a backend returns through the React frontend regardless — mitigated with lockfiles and automated dependency auditing
Django admin exposure	Now sits next to a public API on shared infrastructure and needs its own access restriction (IP allowlist/VPN), not just a login page

Confirmed unaffected by the split: input validation (DRF serializers are a close equivalent to Django Forms); XSS auto-escaping (React escapes by default, the real risk is dangerouslySetInnerHTML without sanitisation, and the API must still validate server-side regardless since it is a directly callable surface independent of what the "trusted" frontend sends); and response-header-level protections (clickjacking, HSTS/SSL-redirect settings) which apply identically regardless of rendering model.

5.7 Authentication Strategy: Session-cookie + CSRF

Criterion	JWT	Session-cookie + CSRF
Ecosystem onboarding cost	Adds new concepts — signing, claim validation, refresh rotation — on top of an already-adopted framework	Wins — the default Django approach
Security	Real, unsolved storage problem: localStorage is XSS-exploitable; an httpOnly cookie avoids that but reintroduces CSRF, defeating JWT's main argued advantage	Wins — an httpOnly cookie is immune to XSS-based theft by construction
Revocation	Stateless by design — cannot kill an active token before expiry without a server-side blocklist	Wins — deleting the session row kills access immediately, which matters for officer-termination or compromised-credential scenarios

> **DECISION:** **Session-cookie + CSRF. "JWT avoids CSRF" only holds if the token is stored somewhere CSRF cannot reach, which means accepting XSS exposure instead — a worse trade for a government system. Instant, server-controlled revocation is the deciding operational factor.**

5.8 Database: Neon (Managed PostgreSQL)

The initial assumption was that Supabase was the only viable free PostgreSQL option; this was checked directly and corrected through research.

Candidate	Free-tier reality	Verdict
Render	Free Postgres deleted permanently	Eliminated

	after 30 days, no backups, no high availability	
Supabase	500MB database, but free projects pause after 7 days of inactivity and require manual dashboard intervention to resume; no automated backups on the free tier	Real contender, lost on idle-recovery behaviour
Neon	100 CU-hours/month free, built-in connection pooling (up to 10,000 connections) on all plans, backed by a ~$1B 2025 acquisition by Databricks	Selected

The deciding factor was idle behaviour, not price — both are free. Supabase requires someone to notice a paused project and manually restore it; Neon scales to zero and resumes automatically on the next query (a 300–800ms cold start). For a government system that needs to look reliably available even in its earliest deployment phase, “down until someone notices and logs in” is a real liability that “brief delay on first request” is not. Supabase's bundled platform features (auth, storage, instant APIs) would also sit unused, since Django+DRF already owns those layers — Neon is just PostgreSQL, which is all that is actually needed.

5.9 Object/File Storage: Cloudflare R2

Candidate	Free-tier reality	Verdict
AWS S3	5GB storage, free only for the first 12 months of a new account, then $0.09/GB egress on every download	Eliminated — not a real long-term-free option for a download-heavy government portal
Cloudflare R2	10GB storage, permanent (not time-limited), zero egress fees forever, S3-compatible API	Selected

S3's free tier is a 12-month trial, not a durable feature, and its egress pricing is specifically painful for a document-retrieval-heavy portal where certificates and NOC documents are downloaded repeatedly by applicants and officers. R2's permanent free tier and zero egress policy mean every such download costs nothing instead of metering against a budget that does not exist.

5.10 Supporting Services: CI/CD, Secrets, Email

GitHub Actions was selected for CI/CD without a formal elimination round — 2,000 free Linux CI minutes per month on the Free plan for private repositories is comfortably sufficient at this project's scale, and no second platform offers a structural advantage worth the added surface. Secrets are managed via django-environ and environment variables (a gitignored .env locally, the hosting platform's own injection in production, GitHub Actions' built-in secrets for CI) rather than a dedicated secrets manager such as Vault — that is real infrastructure to run and secure for a problem this system's scale does not yet have, and one that costs money, conflicting with the free-tier constraint set everywhere else in this stack.

Candidate (email)	Current reality	Verdict
SendGrid	Permanent free tier discontinued in 2025; now a 60-day trial only, then a paid minimum	Eliminated
AWS SES	Cheapest at scale, but starts in sandbox mode requiring manual production-access approval, and needs hand-built bounce/complaint/monitoring infrastructure	Eliminated — same reasoning that eliminated Celery+Redis: real ongoing operational burden for a problem this system's modest volume doesn't justify
Resend	3,000 emails/month free, permanently, no credit card required, minimal setup	Selected

6. Domain-Specific Engineering Decisions

The decisions in this section sit at the boundary between regulatory requirement and engineering judgment: the law or regulation determines what is permitted, but how to implement it within that boundary remains a real engineering choice with real alternatives.

6.1 Aadhaar Handling and Identity Verification

Option	Requirement	Verdict
Build an own Aadhaar Data Vault	HSM-backed encrypted vault meeting UIDAI's technical bar, plus a dedicated security audit — effectively becoming a fully compliant AUA/KUA-equivalent entity	Eliminated — an institutional undertaking disproportionate to this project's scope
Route through an existing licensed AUA/KUA	Sub-AUA onboarding — contractual, usually a paid commercial relationship	Eliminated — doesn't fit the project's timeline or budget
UIDAI Offline Paperless KYC / Secure QR verification	Verify UIDAI's own digital signature on the applicant's offline e-KYC data or Secure QR code, using UIDAI's public certificate — no live API call to UIDAI's database, no AUA/KUA licence needed	Selected

This mechanism is specifically what UIDAI built for organisations seeking genuine identity assurance without full AUA/KUA licensing. Only a salted hash of the Aadhaar number, for deduplication, and the last four digits, for display, are ever stored — never the full number, anywhere persistent. This is not a best-practice suggestion: the Aadhaar Act (Sections 37, 38, 42) carries personal criminal liability for mishandling, independently confirmed against current sources during this project's research.

6.2 Digital Signature / Certificate Issuance

A Class-3 DSC token costs approximately ₹1,500–3,000 per year per officer from a CCA-licensed Certifying Authority. This is stated plainly because it is a regulatory cost MbPA must bear institutionally — it cannot be engineered away through a free-tier substitution.

Option	Verdict
Server holds officers' private keys and signs on their behalf	Eliminated — defeats the legal purpose of an individually attributable signature; a server compromise would mean forged government certificates
Local signing (officer's own DSC token + browser-side signer) → signed PDF uploaded back, signature verified on receipt	Selected — the proven pattern already used for GST, MCA, and Income Tax e-filing in India, and consistent with MIDC BPAMS's DSC-only model (§3.3)
Aadhaar eSign (cloud-hosted key, OTP-authenticated per signature)	Named as a real alternative worth raising with MbPA — no physical token required, pay-per-signature instead of an annual cost — not architected

	around exclusively

Django generates the unsigned PDF certificate via pyHanko, the officer signs it locally, and the system verifies the PKCS#7/CAdES signature on receipt before treating the certificate as final. This verification step is what makes the architecture agnostic to which signing mechanism produced the signature. Given the timeline, the V1 fallback is manual sign-print-scan-upload through the identical receive-and-verify code path — real one-click DSC-token signing is a fast-follow, not a blocker to shipping.

6.3 DPDP Act, 2023 — Privacy Design

- Notice, not consent, for the core function. Section 7(b) treats government processing for the provision of any certificate, licence, or permit as a legitimate use not requiring consent. A plain privacy notice replaces a consent-gated application form, though notice, purpose-limitation, and security obligations under the DPDP Rules' Second Schedule still apply.
- Retention beats erasure for application records. Section 12 grants correction, completion, and erasure rights, but erasure applies only where retention is not otherwise legally required — and the Public Records Act, 1993 mandates retention for statutory government records. Correction of applicant details is supported; self-service deletion of application records is not, a distinction documented explicitly so it is never ambiguous later.
- Breach notification: Section 8(6) and Rule 7 require notifying the Data Protection Board and affected Data Principals without delay, with a detailed report to the Board within 72 hours. This is a process runbook, not code — but the audit-logging system (§6.5) is what makes scoping a breach within 72 hours operationally achievable.
- Phasing: DPDP Rules were notified 14 November 2025; Consent Manager provisions phase in from 13–14 November 2026; full substantive compliance obligations apply from 13–14 May 2027. Designing for this now, ahead of the deadline, is the deliberate choice.
- Open, pending MbPA: the identity of the actual Data Protection Officer/grievance contact to list on the portal — an organisational answer, not an engineering one.

6.4 Milestone Lifecycle — Implementation Pattern

This covers how the engine is built, not what the rules say — the actual SLA day values, zone-specific thresholds, and stream-specific conditions still depend on UPDR-2026, which has not yet been obtained.

Option	Verdict
State-machine library (e.g. django-fsm)	Real branching complexity exists — seven streams, each with a different milestone subset, plus conditional branches such as the demolition step for re-erection or the >50%-BUA conversion rule for additions/alterations — but this means a new dependency with its own decorator-based mental model, adding onboarding cost this project's timeline doesn't favour
Explicit data structure + a single transition-validation service function	Selected

This is consistent with the same “less magic, more explicit” principle applied to the testing-strategy decision below: the actual rules are data — which streams have which milestones, in which order — not complex branching behaviour, and do not need a library to express. Each stream's milestone sequence is

a plain data structure; one service function validates every transition against it, plus a deemed_clearance_eligible flag hardcoded false for the Occupancy Certificate milestone.

> **DECISION: The Occupancy Certificate milestone is excluded from deemed-clearance by design, not by oversight. Comparative Indian Ease-of-Doing-Business law is unusually consistent on this point — even TS-bPASS, the most aggressive self-certification regime examined in §3.2, treats deemed approval as revocable and provisional, never as a substitute for the final safety determination. A wrong default here could mean a building receiving occupancy sign-off with no inspection ever having occurred.**

6.5 Audit Logging and Records Compliance

Candidate	Verdict
django-easy-audit	Eliminated — lightly maintained, last release in 2024, reported migration issues
django-pghistory	Eliminated — database-trigger-based, technically the strongest option, but trigger-level complexity is a higher onboarding cost than this project's timeline favours
django-auditlog	Close second — actively maintained, JSON field-diffs
django-simple-history	Selected — actively maintained, full per-model history, integrates directly with Django admin, already the locked-in internal MbPA ops tool

django-simple-history handles generic model-change tracking; a deliberate, purpose-built AuditEvent log captures officer decisions specifically (approve/reject/IOD-issue — who, when, what, why). The library answers “what changed on this row”; the legally significant moments for RTI and audit purposes get an explicit semantic record on top of that, satisfying the Public Records Act's integrity expectations.

6.6 Integration Points

The conditional-NOC checklist (Railway, CRZ/MCZMA, MHCC heritage, AAI/aviation, MPCB) — the wizard discovered in the prototype's code but undocumented in its PRD (§2.3) — is implemented as a self-attested checklist with document upload for the first version, stored behind an interface that a real API check (such as AAI's coordinate-based NOCAS lookup) could fill in later without restructuring the surrounding application flow. This is a deliberately deferred integration, not a forgotten one. Live payment gateway and GIS/mapping integration remain explicitly out of scope, consistent with the source PRD.

7. Data Architecture

The data model translates every decision above into twenty-one concrete entities, each stress-tested with adversarial “how does this break” thinking before being locked, rather than accepted on first draft. The full schema is maintained as a companion Data Requirements Document; this section captures the findings significant enough to belong in the engineering record itself.

7.1 Entity Overview

Nineteen core entities plus two added directly as a result of adversarial review: User, ApplicantProfile, OfficerProfile, Application, ApplicationParty (added — multi-party filing), Stream, Milestone, MilestoneInstance, DocumentSlot, DocumentUpload, Concession, FeeAssessment, Payment, Certificate, ConditionalClearance, Complaint, AuditEvent, OtpToken, ConfigParameter, plus Holiday (added — working-day SLA calculation).

7.2 Findings That Changed the Schema

Finding	Resolution
Application-number race condition	Generating the sequential portion of an application number by counting existing rows breaks under concurrent requests — the prototype's single-threaded Apps Script model never exposed this, but a multi-worker Django deployment would hit it immediately. Resolved with an atomic database sequence, never a row count.
The fee-snapshot trap	Externalised configuration rates mean a future rate change would otherwise retroactively alter the apparent cost of an already-paid fee. Resolved by snapshotting computed amounts on FeeAssessment plus a reference to which configuration version produced them — immutable once payment begins.
Aadhaar uniqueness vs. hashing strength	A per-row random salt would defeat deduplication entirely; a single static salt is weak against a 12-digit input space (only 10^12 possibilities). Resolved with an application-wide secret pepper held in secrets management, not the database — and flagged as a real open question whether Aadhaar deduplication is even a hard requirement, since dropping it would permit a far stronger hashing scheme.
Audit-log cascade-delete and enforcement gap	Research confirmed that application-level-only immutability “leaves a back door,” and that Django's generic-relation audit fields cascade-delete history when their target is deleted. Resolved with database-level INSERT-only grants on the audit table's role, and a non-cascading reference design so deleting any target never deletes its history.
The combined-clock S1 milestone	PRD §17.1 specifies a single 21-day clock spanning a

	handoff from Estate Officer to Junior Planner. A model assuming one officer per milestone instance would silently reset the clock on handoff. Resolved by attaching the SLA clock to the milestone instance itself, not to whichever officer currently holds it.
Soft-delete is deliberately not uniform	A single shared soft-delete base class — the common Django pattern — was rejected because it would violate at least one of three competing pressures: Certificate and AuditEvent must never be deletable (legal artefacts); OtpToken and erased Aadhaar fields must be hard-deletable (privacy); Application and most children must be soft-deleted (statutory retention). Delete policy is assigned per-entity, justified by which pressure applies.

7.3 What Remains Provisional

ApplicationParty (multi-party filing) and OfficerProfile's zone/stream-specialisation fields are built flexible and left inert, pending MbPA confirmation on co-applicant representation and officer staffing. Designing the more flexible shape now and leaving it dormant is reversible; discovering the need later and retrofitting it onto live data is not.

8. Feasibility — Specified vs. Implemented

Honest scoping is a credibility signal as much as a planning tool. This document represents the complete design phase of the project: every architecture, stack, security, and data-model decision has been made and recorded. No application code has been written against this design yet. Stating this plainly here is more useful to whoever picks this project up next than a built/roadmap table that implies partial implementation exists when it does not.

Phase	Status	Detail
Problem analysis & prototype reconciliation	Complete	Line-by-line review of the existing prototype against its own PRD; eight concrete defects identified (§2.3)
Domain question resolution	Mostly complete	Independent legal/regulatory research closed roughly two-thirds of the open domain questions; the remainder depends on UPDR-2026 text and internal MbPA documents not yet obtained
Technical Design (architecture, stack, security)	Complete	§§4–6 of this document
Data Requirements (schema)	Complete	§7 of this document and its companion DRD
Application code (models, services, views, frontend)	Not started	Next phase — begins with project scaffolding and a direct translation of the DRD into Django models
Testing	Not started	Strategy specified (§9.1); no tests written yet, since no implementation exists to test
Deployment / hosting target	Blocked	NIC/NICSI vs. MeitY-empanelled Government Community Cloud depends on budget and access only MbPA controls

9. Engineering Practice

9.1 Testing Strategy

Candidate	Verdict
pytest-django + factory_boy	Reduces boilerplate via fixtures, but adds two dependencies, and its implicit fixture-injection model is a second testing paradigm layered on top of the Django/DRF stack itself
Django/DRF built-in TestCase / APITestCase + factory_boy	Selected — zero new dependencies beyond what's already chosen; APITestCase is what DRF's own documentation teaches as the default way to test a DRF API

Planned allocation: heavy unit-test coverage on the domain engine (fee calculation, the SLA sweep, milestone transitions); integration tests covering the officer approve/reject workflow end-to-end through the API; a thin end-to-end layer reserved for only the highest-stakes journeys (intake submission, Occupancy Certificate issuance).

9.2 Non-Functional Targets

Target	Value
API response time	Under 500ms, typical
Page load	Under 2 seconds on average broadband
Availability	Best-effort, above 99% once live — no formal SLA at pilot stage, consistent with the underlying free-tier infrastructure carrying no SLA itself
Accessibility	GIGW 3.0 / WCAG 2.1 AA
Browser support	Last 2 versions of evergreen browsers; no Internet Explorer 11
Session timeouts	45 minutes (applicant) / 6 hours (officer), carried from the existing prototype

10. Impact Projection

Quantifying impact before a single line of application code exists is necessarily a projection, not a measurement — it is included because the comparative systems in §3 provide a real evidentiary basis for what a working version of this system should achieve, not because this project has data of its own yet.

Metric	Comparative Evidence	What It Implies for MbPA
Time-to-permit reduction	Mumbai's own construction-permit time fell from 128.5 to 99 days under MCGM's digitisation reforms; Delhi's fell from 157.5 to 91 days	A comparable digitisation effort on port land has direct precedent for a meaningful, not marginal, reduction in time-to-permit
Scale achievable within years, not decades	TS-bPASS processed over 1,00,000 approvals across Telangana within three years of launch	A fully online MbPA system is not a multi-decade adoption curve — comparable systems reach real volume within a single deployment cycle
Informal/unauthorised construction reduction	60–80% of construction in developing economies proceeds without formal approval where the formal process is slow enough to incentivise bypassing it	Faster, more predictable permitting is itself a lever on whether port-land construction happens inside or outside the regulated system

No MbPA-specific application volume, applicant count, or officer-load figures exist yet — these are explicitly flagged in §2 as currently unknown and assumed modest until real usage data says otherwise. Projecting a specific MbPA adoption percentage or absolute case count, in the absence of that baseline, would be a fabricated precision this document deliberately avoids.

11. Regulatory and Compliance Positioning

This system touches more statutory regimes than a typical internal tool, and each one constrains the architecture directly rather than being a documentation afterthought.

Regime	How It Constrains This System
Major Port Authorities Act, 2021	Establishes MbPA's authority as a Special Planning Authority; governs the appeal route for rejected applications via the Adjudicatory Board (Sections 22–32, 54, 58) — though whether a building-permission refusal squarely falls within that Board's stated scope is itself an open question pending legal confirmation
IT Act, 2000	Sections 4–6 give a properly adopted online filing channel and digitally signed documents full legal standing equivalent to paper and wet-ink signatures — the basis for both online submission validity and the DSC-based certificate architecture in §6.2
Aadhaar Act, 2016	Sections 37, 38, and 42 impose personal criminal liability for Aadhaar mishandling — the direct basis for the hash-and-last-4-digits-only storage rule in §6.1
DPDP Act, 2023	Governs notice, consent exemptions, retention, correction, and breach-notification obligations — detailed in §6.3
RTI Act, 2005	MbPA, as a statutory body substantially Centrally funded, is a public authority under Section 2(h); must support proactive disclosure and 30-day response obligations — directly informing the audit-logging design in §6.5
Public Records Act, 1993	Mandates retention of statutory government records, which is the specific reason DPDP's erasure right does not apply to application records (§6.3)
GIGW 3.0 / STQC / CERT-In	Government hosting and accessibility expectations — gov.in/NIC-class hosting, WCAG 2.1 AA accessibility, CERT-In empanelled VAPT before production go-live. These are administrative expectations rather than standalone statute, but deviating from them on a government building-permission system is a real compliance flag, not a style choice.

DECISION: Production go-live with real Aadhaar data and legally binding certificates requires institutional sign-off — STQC certification, CERT-In VAPT, GIGW accessibility audit, empanelled hosting approval — that this document deliberately does not attempt to substitute for. These are MbPA's gates to clear, not engineering deliverables, and are tracked separately in a Production-Readiness handover note.

12. Open Risks and Pending Decisions

Carried forward visibly so that nothing is silently assumed:

- UPDR-2026 text itself — not publicly available; blocks real fee rates, zone-specific FSI/setback/parking benchmarks, document-slot checklists, the 50%/90% stream-conversion thresholds, the demolition-step SLA duration, Temporary Permission renewal terms, and Special Building height/hazard classification criteria.
- Hosting target — NIC/NICSI vs. MeitY-empanelled Government Community Cloud, dependent on MbPA budget and access.
- Data Protection Officer / grievance contact identity — an organisational answer pending from MbPA.
- Appeal routing — whether a rejected application's appeal goes through the Adjudicatory Board or directly to High Court writ jurisdiction.
- IOD auto-generation vs. officer discretion (§2.3) — depends on actual officer workflow preference that cannot be invented in this document.
- Organisational staffing — whether each of the four officer roles is held by a single person or split by geography/stream type.
- Zonal Ready Reckoner Rate source — the Maharashtra IGR e-ASR rate versus a possible MbPA-internal Scale of Rates equivalent for port land.
- Real historical application files, the existing paper SOP, and prescribed government forms — needed before the document-slot checklist and validation rules can be made exhaustive rather than illustrative.
- Exact NOC trigger distances for Railway, MHCC heritage, and MPCB pollution clearances — only the AAI/aviation and CRZ triggers were resolved with concrete, sourced figures during this project's research; the others remain comparative assumptions.
- Whether Aadhaar deduplication is a hard requirement (§7.2) — affects the achievable strength of the Aadhaar hashing scheme.

13. Anticipated Questions

Why was so much of this stack chosen for being free, rather than for being best-in-class?

Because no confirmed institutional budget exists for this project. Every free-tier choice in §5 was still evaluated on technical merit first — Neon was chosen over Supabase for reliability behaviour, not price, since both are free — but candidates that were not genuinely free long-term (AWS S3, SendGrid, AWS SES) were eliminated regardless of their other merits, because a system that depends on infrastructure no one can pay to keep running is not actually a deployable system.

Why does this document recommend Python when Java would serve MbPA's long-term institutional fit better?

It does not hide that tension — §5.1 documents Java + Spring Boot explicitly as the named alternative if the implementation-velocity constraint changes. The honest position is that Java's advantage is about who inherits this system in ten years, a real but future consideration, while Python's advantage is about whether the system gets built at all under the project's actual present constraints.

Why is the Occupancy Certificate excluded from deemed clearance when the rest of the milestone chain auto-advances on SLA breach?

Because every comparable Indian system examined treats final occupancy/safety sign-off as the one milestone that is never silently auto-approved — even TS-bPASS, the most aggressive self-certification system in this comparison, makes deemed approval explicitly revocable and conditions it on accountable self-certification rather than silence. This is a life-safety default, not a conservative engineering preference.

Why does this document not claim the system is compliant, certified, or production-ready?

Because it is not, and a handoff document that overstated its own completeness would actively mislead whoever picks this project up next. §8 states plainly that zero application code exists yet; §11 states plainly that STQC, CERT-In, and GIGW sign-off are institutional gates this document does not substitute for.

14. References

Government, Policy, and Statutory Sources

- Major Port Authorities Act, 2021 — Government of India
- Information Technology Act, 2000, Sections 4–6 and 15 — Government of India
- Aadhaar (Targeted Delivery of Financial and Other Subsidies, Benefits and Services) Act, 2016, Sections 37, 38, 42
- Digital Personal Data Protection Act, 2023, and DPDP Rules, 2025 (notified 14 November 2025)
- Right to Information Act, 2005, Section 2(h)
- Public Records Act, 1993
- Telangana State Building Permission Approval and Self-Certification System (TS-bPASS) Act, 2020, Section 7
- Mumbai Municipal Corporation Act, 1888, Section 346 — statutory basis for Intimation of Disapproval

Comparative System Sources

- MCGM Online Building Plan Approval System (AutoDCR) — autodcr.mcgm.gov.in, official FAQs and downloads pages
- "TS-bPASS facilitates over a lakh construction approvals in Telangana over 3 years" — Telangana Today, January 2024
- TG-bPASS official portal — tgbpass.telangana.gov.in
- MIDC Building Plan Approval Management System (BPAMS) — Maharashtra Industrial Development Corporation

Economic and Regulatory Research

- World Bank, Doing Business — "Dealing with Construction Permits" archive (programme discontinued 16 September 2021)
- SRCC, "Construction Permit Enablers in India" — Shri Ram College of Commerce
- Sharma, "Ease of Getting Construction Permits in India: Cross Country Comparison with World's Best" (2017)

Technology and Platform References

- Django and Django REST Framework official documentation
- Neon, Cloudflare R2, and Resend official documentation and pricing pages
- django-simple-history, django-auditlog, django-pghistory, django-easy-audit — package documentation and maintenance status
- pyHanko documentation — PDF digital signature generation and verification
- UIDAI circulars on Aadhaar Data Vault requirements (Circular No. 8 of 2025) and Offline Paperless KYC